

AWS SAM-TEMPLATE LEARNING

- 1) Create a directory as SAM-Template
- 2) Run the following command
 - a. `python -m venv myfirstproject`
 - b. `myfirstproject\Scripts\activate`
- 3) run the following command to create the project directory
 - a. `sam init`
- 4) navigate into the project directory
- 5) Run the following 2 command
 - a. `sam build`
 - b. `sam deploy - -guided` (for the 1st time)

Configuring SAM deploy

=====

Looking for config file [samconfig.toml] : Found

Reading default arguments : Success

Setting default arguments for 'sam deploy'

=====

Stack Name [first-lambda]:

AWS Region [us-west-2]:

#Shows you resources changes to be deployed and require a 'Y' to initiate deploy

Confirm changes before deploy [Y/n]:

#SAM needs permission to be able to create roles to connect to the resources in your template

Allow SAM CLI IAM role creation [Y/n]:

#Preserves the state of previously provisioned resources when an operation fails

Disable rollback [y/N]:

HelloWorldFunction has no authentication. Is this okay? [y/N]: y

Save arguments to configuration file [Y/n]:

SAM configuration file [samconfig.toml]:

SAM configuration environment [default]:

- c. `sam deploy` (from the 2nd time onwards)

AWS SAM-TEMPLATE LEARNING

6) Finally, we will receive an output like this

CloudFormation outputs from deployed stack

Outputs

Key HelloWorldFunctionIamRole

Description Implicit IAM Role created for Hello World function

Value arn:aws:iam::174303133794:role/first-lambda-HelloWorldFunctionRole-jAVgSDNXyvuV

Key HelloWorldApi

Description API Gateway endpoint URL for Prod stage for Hello World function

Value https://t8fc6x3ejk.execute-api.us-west-2.amazonaws.com/Prod/hello/

Key HelloWorldFunction

Description Hello World Lambda Function ARN

Value arn:aws:lambda:us-west-2:174303133794:function:first-lambda-HelloWorldFunction-POIMySqlFucp

Now create a simple project using the default configuration

In the path C:\Selva\SAM-Template\first-lambda\first-lambda

App.py

```
import json

def first_lambda(event, context):
    return {
        "statusCode": 200,
        "body": json.dumps({
            "message": "AWS Lambda is super cool"
        }),
    }
}
```

AWS SAM-TEMPLATE LEARNING

C:\Selva\SAM-Template\first-lambda

Template.yaml

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: >
  first-lambda

Sample SAM Template for first-lambda

Resources:
  HelloWorldFunction:
    Type: AWS::Serverless::Function # More info about Function Resource:
    https://github.com/awslabs/serverless-application-model/blob/master/versions/2016-10-31.md#awsserverlessfunction
    Properties:
      CodeUri: first-lambda/
      Handler: app.first_lambda
      Runtime: python3.13
```

Run the following:

```
sam build
sam deploy
```

first-lambda-HelloWorldFunction-MWEhV79qfD51

 This function belongs to an application. [Click here](#) to manage it.

▼ Function overview [Info](#)

Diagram | Template



**first-lambda-
HelloWorldFunction-
MWEhV79qfD51**



Layers

(0)

[+ Add trigger](#)

[+ Add destination](#)

AWS SAM-TEMPLATE LEARNING

<https://docs.aws.amazon.com/lambda/latest/dg/python-context.html>

datatypes.py

```
import time
import os
import random

global_random_val = random.random()

def cold_start_basics(event, context):
    local_random_val = random.random()
    print(local_random_val)
    print(global_random_val)

def simple_types(event, context):
    print(event)
    return event

# {
#   "key": ["john", "bob"]
# }
def list_types(event, context):
    print(event["key"])
    student_scores = {"john":100, "bob": 90, "bharath": 100}
    scores = []

    for name in event["key"]:
        scores.append(student_scores[name])

    return scores

# {
#   "key": {"john": [10,20,30], "bob": [40,50,60]}
# }
def dict_types(event, context):
    john_scores = event["key"]

    for score in john_scores["john"]:
        print(score)

    return event

def lambda_handler(event, context):
```

AWS SAM-TEMPLATE LEARNING

```
#PRINT THE CONTEXT
print("Lambda function ARN:", context.invoked_function_arn)
print("CloudWatch log stream name:", context.log_stream_name)
print("CloudWatch log group name:", context.log_group_name)
print("Lambda Request ID:", context.aws_request_id)
print("Lambda function memory limits in MB:", context.memory_limit_in_mb)
# We have added a 1 second delay so you can see the time remaining in get_remaining_time_in_millis.
time.sleep(7)
print("Lambda time remaining in MS:", context.get_remaining_time_in_millis())
print("restapiurl ::: ",os.getenv('restapiurl'))
print("dbname ::: ",os.getenv('dbname'))
```

template.yaml

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: >
  first-lambda
  Sample SAM Template for first-lambda

Resources:
  HelloWorldFunction:
    Type: AWS::Serverless::Function # More info about Function Resource:
    https://github.com/awslabs/serverless-application-model/blob/master/versions/2016-10-31.md#awsserverlessfunction
    Properties:
      CodeUri: first-lambda/
      Handler: datatypes.cold_start_basics
      Runtime: python3.13
      Timeout: 10
      Environment:
        Variables:
          restapiurl: http://dummyurl
          dbname: mydb
```

How to execute the command from local:

```
aws lambda invoke --invocation-type Event --function-name firstlambda-HelloWorldFunction-4G2K4HGOZQX0 outputfile.txt //Asynchorous

aws lambda invoke --invocation-type RequestResponse --function-name versionTest outputfile.txt //Synchorous
```

```
aws lambda invoke --invocation-type RequestResponse --function-name <FunctionName> outputfile.txt
```

Synchronous Flow Using Rest Api (AWS API Gateway)

USCASE: Order Item Use case:

- 1) Create lambda function: Create Order Lambda
- 2) Create lambda function: Read Order Lambda
- 3) API gateway: Api gateway for both Create and Read Order

Create.py

```
import json
import boto3
import os

def lambda_handler(event, context):
    order=json.loads(event['body'])
    dynamodb=boto3.resource('dynamodb')
    table_name=os.environ.get('ORDERS_TABLE')
    table=dynamodb.Table(table_name)
    response=table.put_item(TableName=table_name,Item=order)
    print(response)
    return {
        'statusCode': 201,
        'headers': {},
        'body': json.dumps(response)
    }
```

Read.py

```
import simplejson as json
import boto3
import os
from boto3.dynamodb.conditions import Key

def lambda_handler(event, context):
    # order = {'id': 123, 'itemName': 'Mac Book Pro', 'quantity': 100}
    dynamodb=boto3.resource('dynamodb')
    table_name=os.environ.get('ORDERS_TABLE')
```

AWS SAM-TEMPLATE LEARNING

```
table=dynamodb.Table(table_name)
order_id= int(event['pathParameters']['id'])
response=table.query(KeyConditionExpression=Key('id').eq(order_id))

return {
    'statusCode': 201,
    'headers': {},
    'body': json.dumps(response['Items'])
}
```

Requirements.txt

```
requests
boto3
simplejson
```

Template.yaml

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: >
  orders-api

Globals:
  Function:
    Runtime: python3.13
    Timeout: 30
  Architectures:
    - x86_64
  Environment:
    Variables:
      ORDERS_TABLE: !Ref OrdersTable

Resources:
  OrdersTable:
    Type: AWS::Serverless::SimpleTable
    Properties:
      PrimaryKey:
        Name: id
        Type: Number
  CreateOrderFunction:
```

AWS SAM-TEMPLATE LEARNING

Type: AWS::Serverless::Function # More info about Function Resource:
<https://github.com/awslabs/serverless-application-model/blob/master/versions/2016-10-31.md#awsserverlessfunction>

Properties:

CodeUri: orders_api/

Handler: create.lambda_handler

Events:

CreateOrders:

Type: Api

Properties:

Path: /orders

Method: POST

Policies:

- DynamoDBCrudPolicy:

TableName: !Ref OrdersTable

ReadOrderFunction:

Type: AWS::Serverless::Function # More info about Function Resource:
<https://github.com/awslabs/serverless-application-model/blob/master/versions/2016-10-31.md#awsserverlessfunction>

Properties:

CodeUri: orders_api/

Handler: read.lambda_handler

Events:

ReadOrders:

Type: Api

Properties:

Path: /orders/{id}

Method: GET

Policies:

- DynamoDBReadPolicy:

TableName: !Ref OrdersTable

Outputs:

CreateOrdersAPI usage of the !Sub and the !GetAtt

CreateOrdersAPI:

Description: "API Gateway endpoint url for creating orders"

Value: !Sub "https://\${ServerlessRestApi}.execute-api.\${AWS::Region}.amazonaws.com/Prod"

CreateOrderFunction:

Description: "Create Order Function ARN"

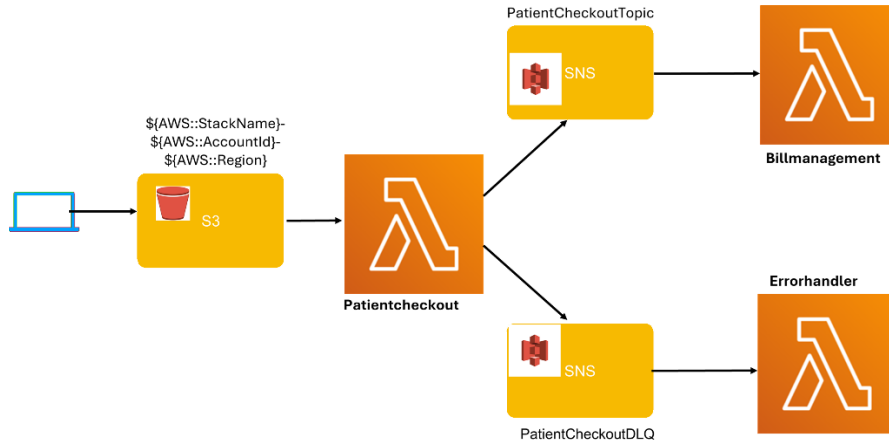
Value: !GetAtt CreateOrderFunction.Arn

CreateOrderFunctionIamRole:

Description: "Create Order Function IAM Role ARN"

Value: !GetAtt CreateOrderFunctionRole.Arn

Asynchronous Flow using the SNS & SQS



InputJson File to be uploaded to the S3 bucket

S3 bucket name: `${AWS::StackName}-${AWS::AccountId}-${AWS::Region}`

```
[
  {
    "firstName":"Bob",
    "middleName":"BobM",
    "lastName":"BobL",
    "ssn": "123456789"
  },
  {
    "firstName":"Alice",
    "middleName":"AliceM",
    "lastName":"AliceL",
    "ssn": "123456781"
  }
]
```

Template.yaml

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: >
  patientcheckout
```

AWS SAM-TEMPLATE LEARNING

Globals:

Function:

Timeout: 30

Runtime: python3.13

Architectures:

- x86_64

Resources:

ClaimManagementQueue:

Type: AWS::SQS::Queue

PatientCheckoutDLQ:

Type: AWS::SNS::Topic

PatientCheckoutTopic:

Type: AWS::SNS::Topic

PatientCheckoutBucket:

Type: AWS::S3::Bucket

Properties:

BucketName: !Sub \${AWS::StackName}-\${AWS::AccountId}-\${AWS::Region}

PatientCheckoutFunction:

Type: AWS::Serverless::Function # More info about Function Resource:

<https://github.com/awslabs/serverless-application-model/blob/master/versions/2016-10-31.md#awsserverlessfunction>

Properties:

CodeUri: patient_checkout/

Handler: patientcheckout.lambda_handler

DeadLetterQueue:

Type: SNS

TargetArn: !Ref PatientCheckoutDLQ

Policies:

- SNSPublishMessagePolicy:

TopicName: !GetAtt PatientCheckoutTopic.TopicName

- S3ReadPolicy:

BucketName: !Sub \${AWS::StackName}-\${AWS::AccountId}-\${AWS::Region}

Environment:

Variables:

PATIENT_CHECKOUT_TOPIC: !Ref PatientCheckoutTopic

Events:

S3Event:

Type: S3

Properties:

Bucket: !Ref PatientCheckoutBucket

Events: s3:ObjectCreated:*

BillManagementFunction:

AWS SAM-TEMPLATE LEARNING

Type: AWS::Serverless::Function # More info about Function Resource:
<https://github.com/awslabs/serverless-application-model/blob/master/versions/2016-10-31.md#awsserverlessfunction>

Properties:

CodeUri: patient_checkout/

Handler: billmanagement.lambda_handler

Events:

SNSEvent:

Type: SNS

Properties:

Topic: !Ref PatientCheckoutTopic

ErrorHandlerFunction:

Type: AWS::Serverless::Function # More info about Function Resource:
<https://github.com/awslabs/serverless-application-model/blob/master/versions/2016-10-31.md#awsserverlessfunction>

Properties:

CodeUri: patient_checkout/

Handler: errorhandler.lambda_handler

Events:

SNSEvent:

Type: SNS

Properties:

Topic: !Ref PatientCheckoutDLQ

ClaimManagementFunction:

Type: AWS::Serverless::Function # More info about Function Resource:
<https://github.com/awslabs/serverless-application-model/blob/master/versions/2016-10-31.md#awsserverlessfunction>

Properties:

CodeUri: patient_checkout/

Handler: claimmanagement.lambda_handler

Events:

ClaimEvent:

Type: SQS

Properties:

Queue: !GetAtt ClaimManagementQueue.Arn

BatchSize: 10

Patientcheckout.py

```
import boto3
import json
import os
import logging
```

AWS SAM-TEMPLATE LEARNING

```
s3 = boto3.client('s3')
snsclient = boto3.client('sns')
logger = logging.getLogger('patientcheckout')
logger.setLevel(logging.INFO)

def lambda_handler(event, context):
    topic = os.environ.get('PATIENT_CHECKOUT_TOPIC')
    bucket_name = event['Records'][0]['s3']['bucket']['name']
    #print("Bucket name : ",bucket_name)
    file_key = event['Records'][0]['s3']['object']['key']
    logger.info('Reading {} from {}'.format(file_key, bucket_name))
    #print("file key name : ",file_key)
    obj = s3.get_object(Bucket=bucket_name, Key=file_key)

    file_content= obj['Body'].read().decode('utf-8')
    checkout_events=json.loads(file_content)

    #print("checkout events : ",checkout_events)

    for each_event in checkout_events:
        logger.info('Message being published')
        logger.info(each_event)
        #print(each_event)
        snsclient.publish(
            TopicArn=topic,
            Message=json.dumps({'default':json.dumps(each_event)}),
            MessageStructure='json'
        )

    #snsclient.publish(
    #    TopicArn=topic,
    #    Message=json.dumps(each_event),
    #    MessageStructure='json'
    #)
```

Billmanagement.py

```
def lambda_handler(event, context):
    message = event['Records'][0]['Sns']['Message']
    print(message)
```

AWS SAM-TEMPLATE LEARNING

Errorhandler.py

```
def lambda_handler(event, context):  
    message = event['Records'][0]['Sns']['Message']  
    print(message)
```

Requirements.txt

```
requests  
boto3  
json
```

claimmanagement.py

```
def lambda_handler(event, context):  
    for each_record in event['Records']:  
        print(each_record['body'])
```

